

Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

**Master of Science
in
Artificial Intelligence**

Your Name

Submitted on: February 6, 2026

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1 Introduction

1.1 Motivation

Scalable Vector Graphics (SVG) play an essential role in modern digital design due to their scalability, compactness, and support for semantic editing. Unlike raster images, SVGs describe visual content through structured, text-based geometry using paths, shapes, and hierarchical groupings, allowing for precise control, reusability, and easy modification.

In recent years, machine learning and deep learning models have been applied to the generation of SVGs from various input modalities, such as text prompts, raster images, and sketches. Open-source research has produced a variety of approaches, including sequence models, transformer-based architectures, and differentiable optimization methods, each proposing different strategies for representing and producing vector graphics.

Given this diversity, it becomes relevant to study how these open-source models differ in their architectures, internal representations, and the structure of the SVGs they generate. Understanding these aspects can provide a clearer picture of how SVGs are represented and produced computationally, and what factors may influence the quality and editability of the resulting graphics.

1.2 Problem Statement

While multiple open-source models exist for SVG generation, they employ heterogeneous approaches in terms of architecture, data representation, and output structure. Some models treat SVGs as sequences of drawing commands, others as hierarchical objects, and still others as continuous geometric parameters optimized through differentiable rendering.

This diversity makes it challenging to compare how different approaches handle SVG generation and how these choices influence the final output. Furthermore, the evaluation

of SVG quality often depends on various factors, such as path simplicity, structural organization, and editability, which are not always measured in the same way.

Therefore, the problem addressed in this research is to systematically describe, compare, and analyze open-source SVG generation methods, focusing on their architectures, internal representations, and output characteristics, in order to understand how these design choices relate to the quality and structure of the generated SVGs.

1.3 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.
2. To collect, review, and categorize existing open-source methods for SVG generation across various input modalities.
3. To investigate the types of deep learning and machine learning architectures employed in SVG generation models and analyze their design principles.
4. To examine how these architectures represent SVG data internally and how their outputs are transformed into standard SVG files.
5. To explore and evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.
6. To assess and compare the output quality of current SVG generation models based on the identified evaluation criteria.

1.4 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?
2. What open-source methods currently exist for generating SVGs from different input modalities?
3. What types of deep learning and machine learning architectures are models used for SVG generation?

4. What kinds of structure do these models produce?
5. How can the quality and editability of generated SVG be evaluated?
6. What is the observed quality of SVG outputs across different models?
7. What makes open source model suitable for testing?
8. Which open-source SVG generation models are most suitable for testing and comparison?
9. How do the selected open-source SVG generation models perform in terms of defined criteria (eg. editability, quality and practical usage)?

1.5 What is contribution of the thesis to the field?

This thesis proposes a structured view of SVG generation and vectorization methods by classifying them according to how SVGs are produced at inference time, and examines how these design choices affect editability and practical usability.

2 Systematic Literature Review

Kitchenham’s guidelines were originally developed for evidence-based software engineering and have become one of the most widely adopted standards for conducting systematic literature reviews in computer science and technical research disciplines [1]. The methodology provides detailed, domain-specific procedures for planning, searching, selecting, and synthesizing studies in areas where research outputs are often heterogeneous—such as algorithmic models, machine learning architectures, or engineering pipelines. Unlike biomedical-focused protocols such as PRISMA [2], which were designed primarily for clinical trials and human-subject research with standardized quantitative outcomes, Kitchenham addresses challenges unique to computational research: diversity of study designs, inconsistent terminology, non-uniform evaluation metrics, and the frequent presence of code-based or algorithmic artifacts instead of comparable statistical datasets. Similarly, frameworks like SALSA [3] emphasize high-level structuring but do not prescribe the step-by-step operational procedures needed for reproducibility in technical fields.

For this thesis—situated within deep learning, machine learning, and vector-graphics generation—a Kitchenham-style SLR is needed because the literature consists of diverse model architectures, varied input modalities, and different evaluation criteria. Kitchenham’s protocol supports this complexity by providing structured phases (planning, conducting, and reporting) that guide the reviewer through formulating rigorous research questions, defining search strategies, conducting selection criteria, assessing quality, data extraction and performing narrative synthesis. This structure minimizes bias, increases reproducibility, and ensures that the review captures the full range of relevant deep-learning and vectorization approaches.

2.1 Phase 1 — Planning

2.1.1 Review Protocol

Databases

- arXiv
- Google Scholar
- ACM Digital Library

Search Strings/Keywords

SVG Generation

"SVG generation"
"scalable vector graphics" AND generation
"SVG generative models"
"vector graphics generation"
"SVG generation" AND deep learning
"SVG generation" AND machine learning

Raster-to-Vector Vectorization

"raster-to-vector" AND deep learning
"image-to-vector" AND neural network
"image vectorization" AND machine learning
"deep vectorization"
"differentiable vector graphics"

Text-to-SVG

"text-to-SVG"
"text-to-vector graphics"

Structure of SVG, Editability and Quality

"SVG Basics"
"SVG fundamentals"
"Scalable Vector Graphics"

"SVG editability"
"editable SVG"
"SVG path simplification"
"layered SVG" AND generation"

Survey and Review Searches

"SVG" AND "systematic literature review"
"vector graphics" AND survey
"vector image" AND review
"image vectorization" AND survey
"graphic generation" AND review
"SVG Generation" AND benchmark
"image vectorization" AND benchmark

Study Selection Criteria

Inclusion Criteria:

- **Language and Publication:** Written in English and published in a peer-reviewed journal, conference, or workshop.
- **Time Range:** Published between 2020–2025 (post-emergence of modern SVG generative models such as DeepSVG).
- **Domain Relevance:** Falls within computer vision, computer graphics, AI, machine learning, or deep learning.
- **Reproducibility:** Provides an open-source implementation or references a publicly available repository. Availability of pre-trained weights is preferred, but source code alone is sufficient if the method is reproducible.
- **SVG Output Requirement:** Describes a model, method, or architecture that directly generates SVG or structured vector outputs, including:
 - Bézier curves,
 - SVG `<path>` commands,
 - layered or hierarchical SVG structure,

- vector strokes or SVG-like stroke sequences,
- structured outputs suitable for editability analysis.
- **Input Modalities:** Supports at least one of:
 - image-to-SVG vectorization,
 - text-to-SVG generation,
 - multimodal SVG generation,
- **Architectural Transparency:** Provides sufficient technical detail to extract the model architecture and internal SVG representation (e.g., decoder design, transformer layers, vector tokenization, differentiable rasterization, or geometric parameterization).
- **Evaluation:** Provides quantitative metrics and qualitative SVG examples that allow analysis of structural quality and editability.

Exclusion Criteria:

- **No Vector Output:** The method does not generate vector or SVG outputs (pixel-only image generation).
- **Non-ML Methods:** Purely rule-based or classical algorithmic vectorization methods without machine learning or deep learning.
- **Closed-Source Systems:** No publicly available code, weights or sufficient technical detail for reproduction.
- **Insufficient Evaluation:** Papers lacking SVG evaluation, qualitative vector examples or structural analysis of outputs.
- **Out-of-Scope Vector Work:** Studies focused on vector editing, rendering, compression, markup optimization, or stylistic modification without SVG generation.
- **Domain Mismatch:**
 - handwriting synthesis,
 - font generation,

- datasets,
- sketch generation not intended for SVG graphics.
- **Temporal Exclusion:** Studies published before 2020, except when cited for background context only.
- **LLM Restriction:** Exclude proprietary LLMs that generate SVG text but do not provide architecture, training details, or reproducibility.

Study Selection Procedure

- Initial records
- Duplicate removal
- Title/abstract screening
- Full-text assessment
- Final included studies

Quality Assessment Criteria

Following the recommendations by Kitchenham for evidence-based software engineering [1, 4], each primary study must be assessed for quality, the assessment criteria must be explicitly described, applied consistently, and the overall quality of each study should be reported (e.g., high, medium, or low quality). The four Kitchenham categories used in this review are reporting quality, methodological rigor, reproducibility, and result validity. The criteria are listed below.

The following Kitchenham-style quality criteria were applied to all studies that passed the full-text screening. Each criterion was evaluated as **Yes = 1** or **No = 0**. The total score was then used to classify studies as high, medium, or low quality.

Reporting Quality

- The study states its research aim clearly.
- The model or method architecture is described in enough detail.

- The SVG/vector representation (paths, strokes, Béziers, etc.) is explained.

Methodological Rigor

- The method is appropriate for SVG/vector generation.
- The evaluation procedure is described clearly.

Reproducibility

- The study provides open-source code or enough detail to reproduce results.

Result Validity

- Quantitative metrics or qualitative SVG examples are provided.

Scoring and Classification Each criterion was scored as either **Yes = 1** or **No = 0**. The total score determined the study's quality classification:

- **6–7 points:** High quality
- **4–5 points:** Medium quality
- **0–3 points:** Low quality (may be excluded)

Kitchenham recommends that “studies should be classified as high, medium or low quality based on quality criteria” [1].

Data Extraction Plan According to Kitchenham's guidelines for evidence-based software engineering, the purpose of data extraction is to systematically collect all information required to answer the research questions in a consistent and reproducible way. Kitchenham emphasises that a predefined data extraction form must be created *during the planning phase* to ensure uniformity and reduce subjective variation among reviewers [1].

Predefined Data Extraction Form Kitchenham states that researchers should develop a structured data extraction form before conducting the review. This prevents ad-hoc decisions and ensures that all necessary information is collected consistently across all studies [1].

Alignment with Research Questions Kitchenham notes that extracted data must be driven directly by the review’s research questions. The form should therefore include only fields that support answering the research questions or performing the planned synthesis [1].

Information to Extract The guidelines specify that extraction should capture both bibliographic information and structured study attributes, including study context, aims, methodology, data used, experimental setup, and results. Kitchenham emphasises that the level of detail must be sufficient for later comparison and synthesis [1].

Reviewer Consistency Kitchenham recommends that extracted data should be validated—ideally by a second reviewer—to minimise bias and ensure consistency. If this is not possible, reviewers should re-check their own extraction for consistency [1].

Structured (Not Free-Form) Fields Kitchenham advises the use of structured or semi-structured fields to minimise subjectivity and ensure that comparable information is extracted across studies. Examples include fixed fields for model architecture, datasets, evaluation metrics, and SVG output structure [1].

Avoiding Unnecessary Information Only data needed for answering the research questions or for synthesis should be extracted. Kitchenham warns that recording irrelevant details increases effort and contributes no value to the final review [4].

Ensuring Comparability Across Studies Finally, Kitchenham highlights that the extraction process must ensure that the information gathered from different studies is comparable and suitable for aggregation or narrative synthesis [4]. Using a consistent extraction form enables systematic comparison of SVG generation models.

2.2 Phase 2 — Conducting the Review

2.2.1 Execution of the Search Strategy

The search strings defined in Phase 1 were executed across the selected databases (Google Scholar, arXiv, and ACM Digital Library) to identify relevant literature on SVG generation and vector-based deep learning methods. Searches were performed using the conceptual string groups defined in the search strategy, with database-specific syntax adjustments where necessary. All records retrieved by these searches were exported and documented before any filtering or removal of duplicates. Table ?? summarises the number of studies returned by each search group and database.

Table 2.1: Search Log Summary Across Databases and Conceptual Groups

Search Group	Google Scholar	arXiv	ACM DL	Total
SVG Generation	14	12	4	30
Raster-to-Vector	7	10	2	19
Text-to-SVG	7	6	5	18
Surveys/Benchmarks/Reviews	6	3	1	10
SVG Fundamentals / Background	3	3	2	8
Total Initial findings	37	34	14	85

2.2.2 Removing of Duplications

After merging all search results, duplicate records—including repeated titles and preprint–conference duplicates—were removed. This resulted in 72 unique studies.

2.2.3 Study Selection

- Stage 1: Title, Keywords and Abstract screening.

After removing duplicates, **72** unique studies remained. A brief title and abstract screening excluded works clearly unrelated to SVG generation (e.g., sketch-only models, raster methods, classical vectorization, or closed-source LLMs), resulting in **37** papers proceeding to full-text review.

- Stage 2: Full-text screening

A full-text assessment was conducted on the **37** studies that passed the title and abstract screening. Each paper was examined for reproducibility, architectural transparency, and whether it genuinely produced SVG or structured vector outputs as defined by the inclusion criteria. In total, **10** studies were excluded at this stage: four lacked any open-source implementation, two were benchmark papers evaluating proprietary LLMs such as GPT, Gemini, or Claude, two used non-reproducible LLM-based pipelines without sufficient methodological detail, and one relied on a purely rule-based vectorization approach. Following this eligibility check, **27** studies remained for quality assessment.

Table 2.2: Reasons for Exclusion During Full-Text Screening

Exclusion Reason	Count
No open-source implementation	4
Proprietary LLM benchmark studies (GPT, Gemini, Claude)	2
Non-reproducible LLM-based pipeline	2
Purely rule-based vectorization method	2
Total Excluded	10

Table 2.3: Study Selection Summary Across Screening Stages

Selection Stage	Number of Studies
Initial search results (all databases)	85
After duplicate removal	72
After title and abstract screening	37
After full-text screening	27

3 Background

3.1 Vector Graphics Fundamentals

Images can be represented in different formats, such as raster images (e.g., JPEG, PNG) and vector graphics (e.g., SVG). These formats serve different purposes, and no single format is optimal for all use cases. To understand why vector graphics are important, it is useful to first examine the most common and widely used image format: raster images.

Raster Images: Pixels as the Building Blocks

Raster images are the most widely used image format, especially for photorealistic content. When a photo is taken with a camera, downloaded from the internet, or captured as a screenshot, it is stored as a raster image unless a different format is explicitly chosen. To explain this intuitively, a raster image can be thought of as a grid, similar to a spreadsheet or a Sudoku board, where each cell corresponds to a single pixel. Every pixel stores color information, most commonly in the form of RGB values.

Because raster images may contain millions of pixels, small inaccuracies at individual pixels are usually imperceptible, and neighboring pixels visually compensate for each other. However, this representation also leads to fundamental limitations. Pixels do not carry semantic meaning. For example, if an image contains a table and chairs, moving a chair independently from the table is not possible in a raster image. A pixel does not know whether it belongs to a chair, a table, or the background; it is simply a colored point at a fixed location. The chair is therefore not stored as an object, but as a collection of pixels mixed with pixels from shadows, textures, and surrounding elements. As a result, raster images are difficult to edit in a structured way.

Another limitation arises when raster images are scaled. Raster images have a fixed resolution. When an image is enlarged, new pixels are created through interpolation, meaning the system predicts colors between existing pixels. This process inevitably introduces blur or jagged edges. When an image is reduced in size, pixels are merged or discarded, leading to a loss of detail. In both cases, visual information is lost because the original pixel grid cannot adapt to the new resolution.

Vector Graphics: Shapes Instead of Pixels

Vector graphics take a fundamentally different approach compared to raster images. Instead of storing images as pixels, vector graphics describe images as collections of geometric primitives such as lines, curves, shapes, and text. These primitives are defined mathematically using coordinates and parameters. Using the same table-and-chair example, a vector graphic represents the top, legs, and other parts of the table and chairs as separate shapes or paths. Each component has its own parameters, such as position, size, and color.

Because these components are explicit objects, they can be edited independently. If all parts of a chair are grouped together, the entire chair can be moved, recolored, resized, or removed without affecting the rest of the image. Grouping is discussed in more detail later in this chapter.

This object-based structure also explains why vector graphics scale without loss of quality. When a vector image is resized, the underlying mathematical descriptions are simply interpreted at the new scale. Paths and shapes remain sharp and smooth regardless of resolution, and no interpolation is required. For these reasons, vector graphics are widely used in domains where editability and scalability are important. Graphic and web designers rely on vector formats for logos, icons, and layouts. Data visualization systems use vector graphics to represent charts and plots that must remain clear at different sizes. Game development, user interface design, technical illustration, and marketing materials also heavily depend on vector representations. According to recent W3Techs data from January, SVG has become a web staple, with 65.3% of all websites now relying on it for their graphics.

SVG: A Text-Based Vector Graphics Format

Scalable Vector Graphics (SVG) is a standard format for representing two-dimensional vector graphics. Unlike many other vector formats, SVG is based on XML and stored as plain text. This means that an SVG file is not merely an image, but a readable and editable document describing how the image should be drawn. At its core, an SVG file consists of textual instructions such as drawing a line between two points, defining curves using control points, and filling shapes with specific colors. Even complex shapes are encoded as text-based commands.

This textual structure is one of the main reasons SVG is well suited for computational approaches. SVG files can be generated, parsed, and modified programmatically without relying on opaque binary encodings.

SVG supports different types of graphic elements, including basic shapes (such as lines and circles), text, raster images, and, most importantly, paths. Paths are flexible SVG elements that allow complex shapes to be defined using sequences of commands, such

as moving a virtual pen to a new position, drawing straight lines, and creating smooth curves.

Rather than consisting only of individual shapes, SVG also supports grouping and layered organization of graphic elements. Multiple shapes can be combined into groups that behave as a single unit for styling and editing. For example, the legs and surface of a table can be grouped together so that the entire table can be moved or scaled without affecting its internal structure. This hierarchy improves both human editability and computational manipulation.

Grouping also introduces an implicit structure into SVG documents. Elements are rendered in a defined order, and later elements visually appear on top of earlier ones. As a result, layering becomes part of the vector representation. While many current generative models ignore hierarchical grouping, understanding this structure is important when analyzing the limitations of existing methods and when aiming to produce well-structured vector graphics.

Paths and Bézier Curves

Most SVG graphics are built using paths. A path consists of a sequence of commands such as *move to*, *line to*, and *curve to*. Curved segments are represented using Bézier curves, which are smooth curves controlled by a set of control points.

SVG supports both quadratic and cubic Bézier curves. Quadratic Bézier curves use a single control point and allow the creation of simple, smooth arcs. Cubic Bézier curves use two control points and provide greater flexibility, enabling the representation of more complex shapes. Importantly, these control points do not lie on the curve itself; instead, they influence the direction and curvature of the curve. This parametric representation makes Bézier curves particularly useful, as complex shapes can be adjusted by modifying only a small number of parameters.

From the perspective of this thesis, paths and Bézier curves form the core representation on which many generation and vectorization models operate. Regardless of whether SVG parameters are optimized directly or generated by neural networks, these parameters define the geometry of the final vector graphic.

Coordinate Systems and Transformations

SVG graphics are drawn within a coordinate system defined by a viewport. By default, the origin is located at the top-left corner, with the x-axis pointing to the right and the y-axis pointing downward. SVG allows this coordinate system to be transformed through translation, scaling, rotation, and skewing.

Transformations can be applied to individual shapes as well as to groups of elements.

This allows collections of paths to be moved or resized together while preserving their relative structure. SVG also supports hierarchical coordinate systems, where nested groups define local coordinate frames. This hierarchical structure is especially relevant for complex graphics and layered compositions.

Focus on SVG

While SVG includes many advanced features such as clipping, masking, compositing, and filter effects, these mechanisms primarily affect how graphics are rendered rather than how they are structurally represented. Since the focus of this thesis is on generating and vectorizing geometric structure, such rendering-level effects are not considered further.

In summary, SVG provides a structured, editable, and resolution-independent representation of visual content. Its reliance on explicit geometric primitives, parametric curves, and textual descriptions makes it a natural target for both optimization-based and learning-based approaches to vector graphics generation, which are explored in the following sections.

3.2 Vector Representations

In the previous section, we discussed how vector graphics and are fundamentally different from raster images. SVG graphics are text-based descriptions of geometric shapes defined by mathematical parameters. As the final image, a vector graphic can be seen as a collection of primitives such as lines, curves, paths and shapes.

Based on the chosen models in this paper, vector graphics can be represented internally in three different ways: parametric, sequential, and implicit representations. Should mention that, no matter of how a vector graphic is represented inside a generation or vectorization system, the final output is a SVG file. That SVG file is text, editable and follows the same standard. What changes is not the output format, but the internal representation used by the model during learning or optimization. These internal representations are important because they directly affect what kinds of algorithms can be applied, how stable learning becomes, how editable the result is and how well geometric structure and topology are maintained.

Parametric Representation

The most direct way to represent a vector graphic is through explicit geometric parameters. As the obvious from the name, parametric representations describe vector graphics as collections of shapes, such as lines, curves and paths. Each of them is defined by continuous parameters. These parameters include coordinates, control points, stroke

widths and colors. This representation closely matches how SVG itself is defined. An SVG file can be viewed as a list of shapes and paths, where each element is fully described by a set of numeric parameters. Complex graphics may appear by combining many primitives. Vector graphics are simply structured sets of parameters that can be modified independently.

Parametric representations are highly interpretable and easy to edit manually. Each parameter controls a specific visual property, such as the curvature of a path or the position of a shape, which makes changes predictable. Because of this, parametric representations are commonly used in optimization-based approaches, where vector graphics are refined by directly adjusting geometric parameters to better match a target image or a textual description. At the same time, this representation can be challenging for learning-based models. The number of parameters can be different across graphics and small parameter changes can produce disproportionately large visual effects, making learning less stable. For complex images, explicit vector representations may overfit visual detail by introducing many fragmented paths, leading to cluttered structures that lack semantic coherence.

Sequential Representation

Another way to represent vector graphics is to view them as sequences of drawing commands. In this representation, a vector graphic is encoded as commands, such as moving a pen to a position, drawing a line, or creating a curve. This representation closely follows how paths are written in SVG, with commands and their arguments appearing in sequence. Sequential representations turn vector graphics into a format that looks like a sentence. The order of commands defines drawing order and layering, while the commands themselves describe how the graphic is built step by step. This makes sequential representations well suited for models that operate on autoregressive or sequence-to-sequence models. But, it has also its own limitations. Long command sequences can be difficult to model and small errors early in the sequence may ruin the entire graphic.

w

Implicit Representation

Another approach is to represent vector graphics implicitly using neural networks. In this case, geometry is not stored explicitly as shapes, paths or command sequences. Instead, a neural network represents the graphic by its learned weights. The vector is defined implicitly as a function over continuous space, for example by mapping spatial coordinates to colors, layers or shape indicators. Implicit representations change the focus from explicit geometric primitives to continuous functional descriptions. They provide a fixed-size representation no matter of visual complexity and integrate well with gradient-based optimization. This makes them suitable for models that rely on iterative refinement or semantic supervision from pretrained networks. However, implicit representations are generally less interpretable than others. Because the weights of

the neural network do not map onto specific shapes. Instead, they altogether define a continuous function that determines the properties of the graphic at each spatial location. For this reason, shapes emerge from evaluating this function over space rather than being stored explicitly as separate components. It is one of the main limitations of implicit representations.

3.3 Differentiable Rendering

DiffVG

3.4 Latent Representations of Vector Graphics

Latent Repr.

3.5 Reconstruction-Based Supervision

- Pixel-wise losses (L2, L1) - Perceptual losses - IoU / mask losses

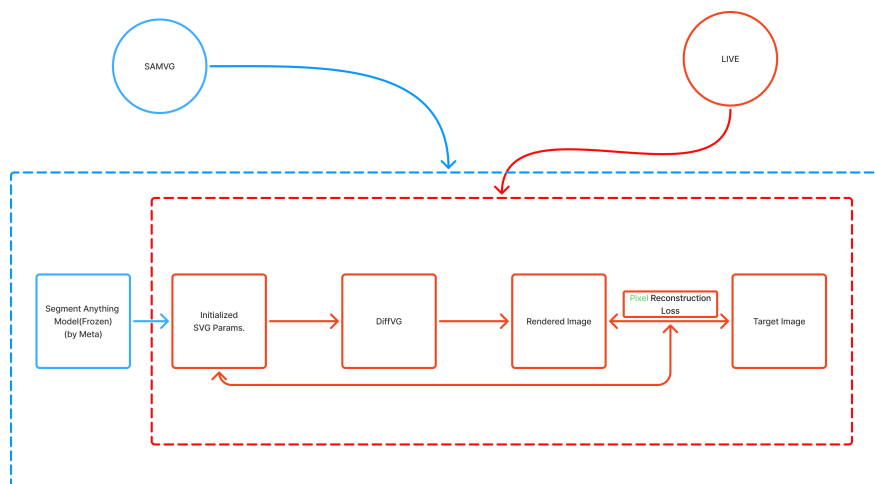
3.6 Semantic Guidance Models

CLIP(Confirmed line item performance) Diffusion Models + Score Distillation

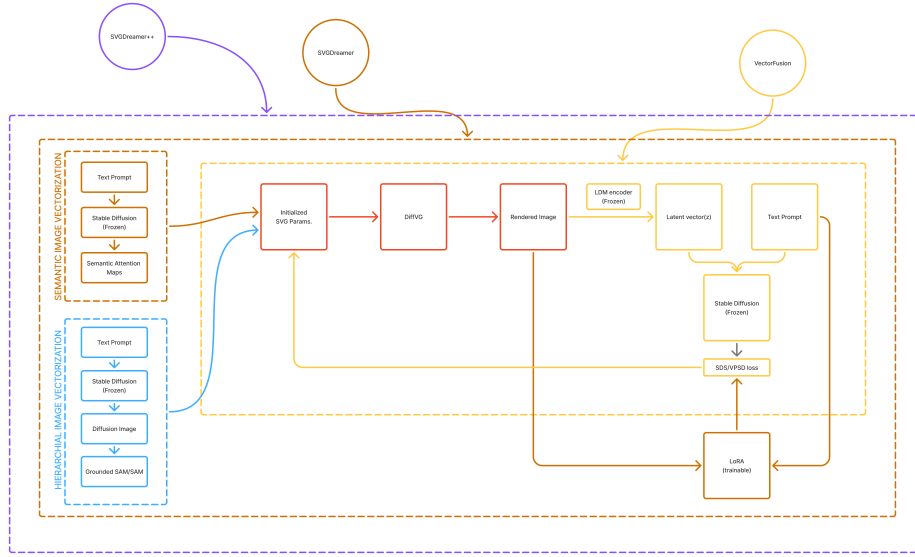
4 Taxonomy of SVG Generation and Vectorization Methods

4.1 Optimziation-based Methods

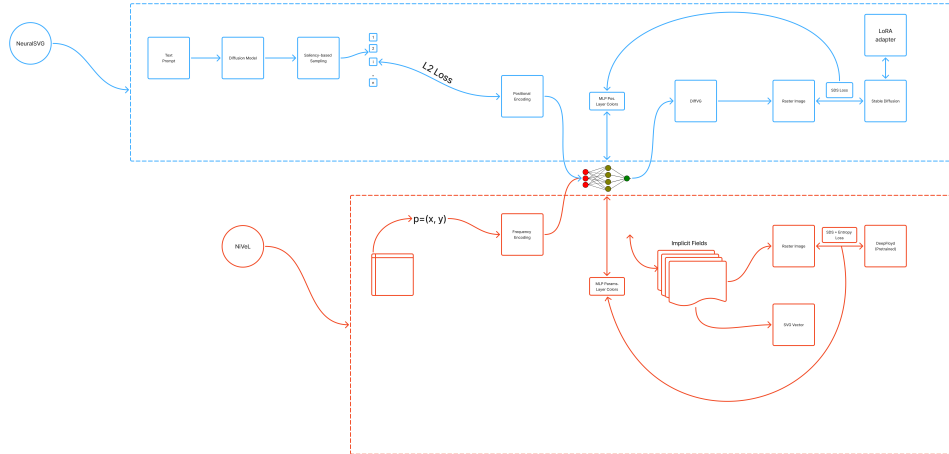
4.1.1 Geometry-Driven Optimization



4.1.2 Semantic-Guided Optimization

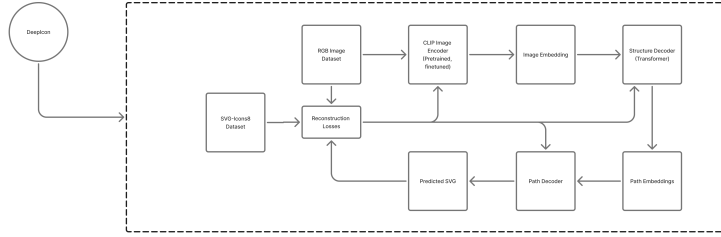
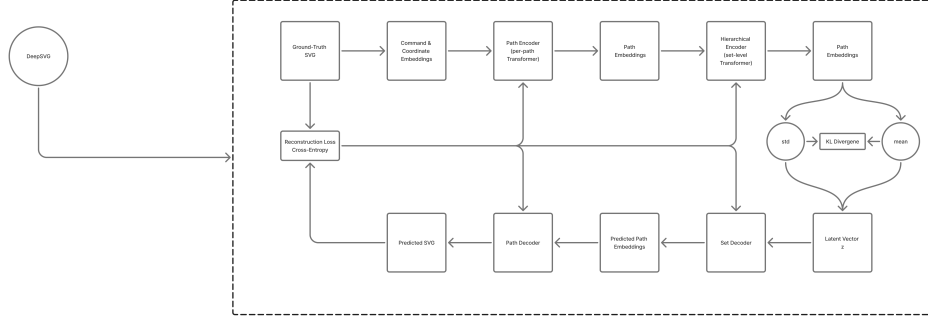


4.1.3 Neural-Parameter Optimization

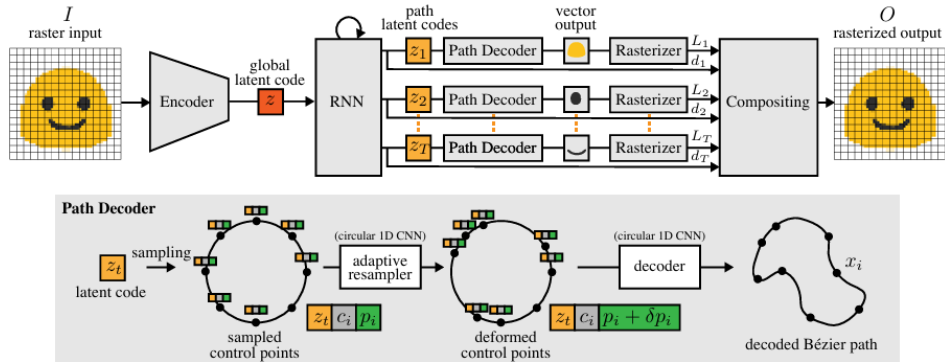


4.2 Feed-Forward Neural Generation

4.2.1 Sequence-Based Generation

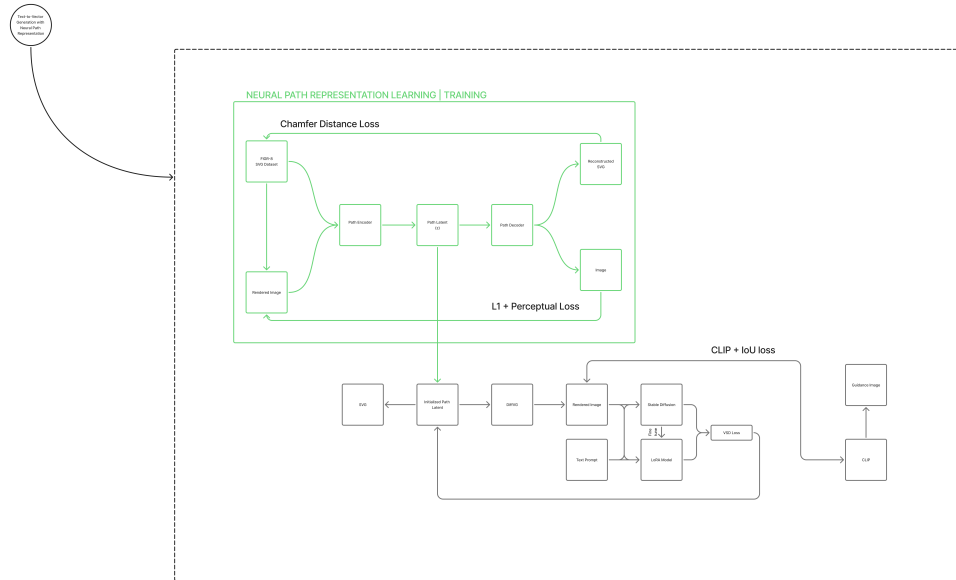


4.2.2 Im2Vec Encoder-Decoder

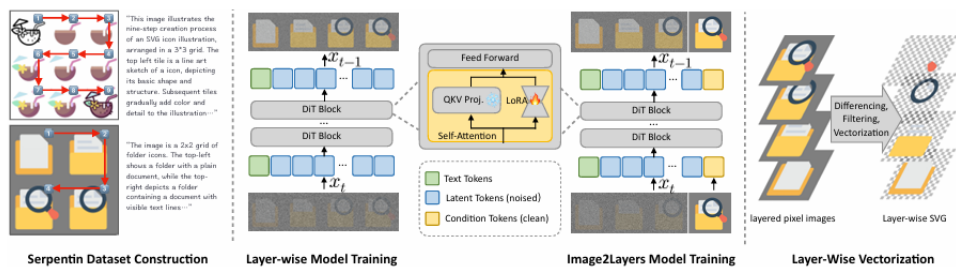


4.3 Hybrid Approaches

4.3.1 Latent Generative Models



4.3.2 LayerTracer



Bibliography

- [1] Barbara Kitchenham. *Procedures for Performing Systematic Reviews*. en. Technical Report. Keele University and NICTA, 2004.
- [2] David Moher, Alessandro Liberati, Jennifer Tetzlaff, Douglas G. Altman. “Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement”. en. In: *BMJ* 339 (July 2009). Publisher: British Medical Journal Publishing Group Section: Research Methods & Reporting, b2535.
- [3] *A typology of reviews: an analysis of 14 review types and associated methodologies - Grant - 2009 - Health Information & Libraries Journal - Wiley Online Library*.
- [4] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, Stephen Linkman. “Systematic literature reviews in software engineering – A systematic literature review”. en. In: *Information and Software Technology* 51.1 (Jan. 2009), pp. 7–15.

List of Figures

Appendix

Great table.

Headline 1	Headline 2
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Table of chosen approaches

Inference Time Overview

Model	Task Type	Inference	Input	Output	Supervision	Core Idea
VectorFusion	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Distills diffusion priors via differentiable rasterization.
SVGFusion	Text-to-SVG	Forward Dec.	Text	Paths	Supervised	Continuous latent space learning using SVGX dataset.
SVGDreamer	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Semantic-driven vectorization (SIVE) and Particle SDS.
SVGDreamer++	Text-to-SVG	Iterative Opt.	Text	Paths	Self-Sup	Hierarchical optimization with adaptive primitive control.
T2V-NPR	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Dual-branch VAE learns path latent space constraints.
LayerTracer	System/Pipeline	Hybrid	Text/Img	Layers/Paths	Supervised	Models designer construction logic via DiT sequences.
Im2Vec	Vectorization	Forward Dec.	Img	Paths	Self-Sup	Differentiable rasterizer allows learning from appearance.
SAMVG	Vectorization	Iterative Opt.	Image	Paths	Opt.-based	Uses SAM for semantic initialization in vectorization.
NIVeL	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Implicit neural layers as optimization domain for SDS.
NeuralSVG	Text-to-SVG	Iterative Opt.	Text	Layers/Paths	Self-Sup	Scene encoded in MLP weights for layered shapes.
LIVE	Vectorization	Iterative Opt.	Image	Layers/Paths	Opt.-based	Layer-wise hierarchical optimization of vector graphs.
DeepSVG	Generation	Forward Dec.	None	Paths	Supervised	Hierarchical Transformer for shape/command disentanglement.
DeepIcon	Vectorization	Forward Dec.	Image	Layers/Paths	Supervised	Hierarchical network for direct image-to-SVG translation.

Training Architecture – Optimization-Based Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
LIVE	SVG parameters	None	Raster-driven	None	Parametric (flat)	None
SAMVG	None	None	Mask-based	None	Parametric (mask-derived)	SAM
VectorFusion	SVG parameters	None	Random / Trace	SDS (Stable Diff.)	Parametric (flat)	Stable Diff.
SVGDreamer	SVG parameters / LoRA	None	Random paths	VPD	Parametric (flat)	Stable Diff.
SVGDream++	SVG parameters / LoRA	None	Random + HIVE	VPD + HIVE loss	Structured parametric	Stable Diff. + SAM
NeuralSVG	MLP weights + colors/ LoRA	None	Saliency-based	SDS	Implicit neural fields	Stable Diff.
NIVeL	MLP weights + layer colors	None	Random weights	SDS	Implicit neural fields	DeepFloyd + CLIP

Training Architecture – Dataset-Trained Methods

Model	Trainables	Training Data	Init.	Supervision	Representation	Priors
DeepSVG	VAE (Transformer) weights	SVGIcons8	Latent sampling	Recon. + KL	Sequential path tokens	None
DeepIcon	Transformer weights	SVGIcons8	CLIP embedding	Reconstruction	Sequential path tokens	CLIP encoder
SVGFusion	VP-VAE + VS-DiT	SVGX	Random latent	Recon. + Denoising	Continuous latent space	CLIP + DINO-v2
T2V-NPR	NPR-VAE + Latents + LoRA	FIGR-8	Random path latents	VSD	Neural Path Repr.	Stable Diff. + CLIP
Im2Vec	Encoder + RNN + 1D CNN	Raster images	Image encoding	Reconstruction	Parametric paths	None
LayerTracer	DiT + LoRA	Serpentine dataset	Diffusion noise	Denoising loss	Layered pixel sequences	Flux DiT + Florence-2

Note: - Order in table: non-learning -i optimization-based -i implicit -i dataset-trained -i systems.

- For optimization-based methods, “trainables” means to per-instance SVG parameters optimized at inference time rather than learned model weights.

- Trained on text prompts means, optimize SVG params at inference time using a frozen pretrained model as semantic prior.

- Semantic distillation means supervision signal got by backpropagating semantic alignments scores from Stable Diff. model to optimize target representation.

- SVGFusion, first trains VAE on SVGX dataset to learn a latent representations of SVGs, then diffusion transformer is trained on those latent repr. + noise. The goal of diff. transformer is to learn to remove the noise. -i if a model learns how to denoise the latent vector, it learns also how to generate new latent vectors.

- A parametric internal representation models SVG graphics as continuous parameters (e.g., Bezier control points and colors) that can be directly optimized with gradients.

- Coarse-to-fine refinement starts from a rough initial set of shapes and then gradually adds paths, repeatedly checking and adjusting them to better match the target description.

- T2V w/NPR: They trained the Neural Path Representation (dual-branch VAE) on the FIGR-8 SVG dataset and it learns: how vector paths map to a latent path space and how to decode latents back into valid vector paths. VSD extends SDS by projecting diffusion-based pixel gradients into the neural path latent space for more stable vector optimization.

- DeepSVG is used by sampling or modifying latent vectors: each random sample produces a new SVG icon, and by interpolating or editing these latent vectors, users can generate variations or blends of existing icons.

Reproducibility Overview

Model	Code	Pre-trained	Hardware	Cost of time	Reproduction Feasibility
LIVE	Yes	No	RTX 3090	2,600–10,000 s	Feasible on high-end consumer GPU (slow).
SAMVG	No	No	RTX 3090	139–2,000 s	Re-implementation possible but requires effort.
VectorFusion	No	No	Not specified	Unknown	Not reproducible due to lack of public implementation.
SVGDreamer	Yes	No	V100 (31GB VRAM)	7–14 min	Requires high-VRAM GPU and LoRA pre-training.
SVGDreamer++	Yes	No	A800 (31GB VRAM)	7 min	Requires high-VRAM GPU and LoRA pre-training.
NeuralSVG	Yes	No	Not stated	Unknown	Hardware requirements unclear.
NIVeL	Pending	No	A100	5 min	High-end GPU required.
T2V-NPR	Yes	VAE (unknown)	RTX 3090	13 min	Feasible on high-end consumer GPU.
DeepSVG	Yes	VAE (unknown)	2×1080Ti (training)	Unknown	Feasible on consumer GPU.
DeepIcon	No	Transformer (unknown)	Not specified	–	Not reproducible (no official code).
SVGFusion	No	VP-VAE + VS-DiT (unknown)	Multi-A800	24–36 s	No official implementation available.
Im2Vec	Yes	VAE (unknown)	Not specified	Unknown	Likely feasible on consumer GPU.
LayerTracer	Yes	DiT + LoRA (unknown)	A100 (80GB)	27–34 s	Enterprise GPU required.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

February 6, 2026, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (<https://my.plagaware.com/>) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

February 6, 2026, Your Name